

CRUD App

Travaux Pratiques — DevSecOps

Node.js · MongoDB · Docker · GitLab CI/CD · Kubernetes · AWS

Auteur : Asma Daoui

Application : API REST CRUD — Node.js + MongoDB

Conteneurisation : Docker (multi-stage build)

Pipeline : GitLab CI/CD — build, test, security, deploy

Cluster local : Docker Desktop Kubernetes (3 pods)

Cloud AWS : EC2 via Terraform

2 juin 2026

Table des matières

1	Application Node.js — src/index.js	2
2	Fichiers de configuration	3
2.1	package.json	3
2.2	.env — Développement local uniquement	4
2.3	.gitignore et .dockerignore	4
3	Docker	4
3.1	Dockerfile — Multi-stage build	4
3.2	Commandes Docker locales	5
4	Kubernetes — Manifests	5
4.1	k8s/configmap.yaml	5
4.2	k8s/mongo.yaml — StatefulSet MongoDB	5
4.3	k8s/deployment.yaml — 3 replicas	6
4.4	k8s/service.yaml — NodePort	7
4.5	Commandes Kubernetes	7
5	GitLab CI/CD — .gitlab-ci.yml	8
5.1	Variable CI/CD requise	10
5.2	GitLab Runner — Installation Windows	10
6	Terraform AWS — terraform/main.tf	10
6.1	Commandes Terraform	12
7	Endpoints API	13
8	Architecture globale	13

1. Application Node.js — src/index.js

API Express avec 5 routes CRUD sur MongoDB : création, lecture (tout/un), modification, suppression. Un endpoint `/health` permet de vérifier que l'app tourne.

Listing 1 – src/index.js

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();
app.use(express.json());
app.use(cors());

// Connexion MongoDB
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch(err => { console.error('MongoDB error:', err.message);
    process.exit(1); });

// Schema
const itemSchema = new mongoose.Schema({
  name: { type: String, required: true },
  description: { type: String, default: '' },
  createdAt: { type: Date, default: Date.now }
});
const Item = mongoose.model('Item', itemSchema);

// Health check
app.get('/health', (req, res) => res.json({ status: 'OK' }));

// CREATE
app.post('/items', async (req, res) => {
  try { res.status(201).json(await Item.create(req.body)); }
  catch (err) { res.status(400).json({ error: err.message }); }
});

// READ ALL
app.get('/items', async (_, res) => {
  res.json(await Item.find().sort({ createdAt: -1 }));
});

// READ ONE
app.get('/items/:id', async (req, res) => {
  try {
    const item = await Item.findById(req.params.id);
    if (!item) return res.status(404).json({ error: 'Not found' });
    res.json(item);
  } catch (err) { res.status(400).json({ error: err.message }); }
});

// UPDATE
```

```
app.put('/items/:id', async (req, res) => {
  try {
    const item = await Item.findByIdAndUpdate(
      req.params.id, req.body, { new: true }
    );
    if (!item) return res.status(404).json({ error: 'Not found' });
    res.json(item);
  } catch (err) { res.status(400).json({ error: err.message }); }
});

// DELETE
app.delete('/items/:id', async (req, res) => {
  try {
    const item = await Item.findByIdAndDelete(req.params.id);
    if (!item) return res.status(404).json({ error: 'Not found' });
    res.json({ message: 'Deleted successfully' });
  } catch (err) { res.status(400).json({ error: err.message }); }
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log('Server -> http://localhost:${PORT}'));
module.exports = app;
```

2. Fichiers de configuration

2.1. package.json

```
{
  "name": "crud-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node src/index.js",
    "test": "jest --forceExit"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "mongoose": "^8.0.0"
  },
  "devDependencies": {
    "jest": "^29.7.0",
    "supertest": "^6.3.3"
  }
}
```

2.2. .env — Developpement local uniquement

Securite

Ce fichier ne doit jamais etre commite sur GitLab. Il est liste dans `.gitignore`. En production, les variables sont passees via Docker `-e` ou Kubernetes ConfigMap.

```
MONGO_URI=mongodb://localhost:27017/cruddb
PORT=3000
```

2.3. .gitignore et .dockerignore

Listing 2 – .gitignore

```
node_modules/
.env
*.log
```

Listing 3 – .dockerignore

```
node_modules
.git
*.log
```

3. Docker

3.1. Dockerfile — Multi-stage build

Build en deux etapes : stage *builder* pour installer les dependances, stage *runtime* pour une image legere en production.

Listing 4 – Dockerfile

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

# Stage 2: Runtime
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY src/ ./src/
COPY package.json ./

EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=5s \
  CMD wget -q0- http://localhost:3000/health || exit 1
```

```
CMD ["node", "src/index.js"]
```

3.2. Commandes Docker locales

```
# Creer le network
docker network create crud-network

# Lancer MongoDB (sans exposer le port, accessible uniquement via
  network)
docker run -d --name mongo --network crud-network mongo:7.0

# Build l'image
docker build -t crud-app:latest .

# Lancer l'app sur le port 4000
docker run -d -p 4000:3000 --name crud-app \
  --network crud-network \
  -e MONGO_URI=mongodb://mongo:27017/cruddb \
  crud-app:latest

# Verifier les logs
docker logs crud-app

# Test
# http://localhost:4000/health
# http://localhost:4000/items
```

4. Kubernetes — Manifests

Deploiement sur Docker Desktop Kubernetes avec 3 replicas pour l'app et 1 StatefulSet pour MongoDB.

4.1. k8s/configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: crud-app-config
  namespace: crud-app
data:
  MONGO_URI: "mongodb://mongo-service:27017/cruddb"
```

4.2. k8s/mongo.yaml — StatefulSet MongoDB

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
  namespace: crud-app
```

```
spec:
  serviceName: mongo-service
  replicas: 1
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
        - name: mongo
          image: mongo:7.0
          ports:
            - containerPort: 27017
          volumeMounts:
            - name: mongo-data
              mountPath: /data/db
      volumeClaimTemplates:
        - metadata:
            name: mongo-data
          spec:
            accessModes: ["ReadWriteOnce"]
            resources:
              requests:
                storage: 1Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
  namespace: crud-app
spec:
  selector:
    app: mongo
  clusterIP: None
  ports:
    - port: 27017
      targetPort: 27017
```

4.3. k8s/deployment.yaml — 3 replicas

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: crud-app
  namespace: crud-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: crud-app
  template:
    metadata:
```

```
labels:
  app: crud-app
spec:
  imagePullSecrets:
    - name: gitlab-registry
  containers:
    - name: crud-app
      image: registry.gitlab.com/asmadaoui-group/crud-app:latest
      ports:
        - containerPort: 3000
      env:
        - name: MONGO_URI
          valueFrom:
            configMapKeyRef:
              name: crud-app-config
              key: MONGO_URI
      readinessProbe:
        httpGet:
          path: /health
          port: 3000
          initialDelaySeconds: 5
          periodSeconds: 10
      livenessProbe:
        httpGet:
          path: /health
          port: 3000
          initialDelaySeconds: 15
          periodSeconds: 20
```

4.4. k8s/service.yaml — NodePort

```
apiVersion: v1
kind: Service
metadata:
  name: crud-app-service
  namespace: crud-app
spec:
  selector:
    app: crud-app
  type: NodePort
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30080
```

4.5. Commandes Kubernetes

```
# Deployer tous les manifests
kubectl apply -f k8s/ -n crud-app

# Verifier les pods (3 crud-app + 1 mongo)
kubectl get pods -n crud-app

# Verifier les services
```

```
kubectl get svc -n crud-app

# App disponible sur
# http://localhost:30080/health
# http://localhost:30080/items
```

5. GitLab CI/CD — .gitlab-ci.yml

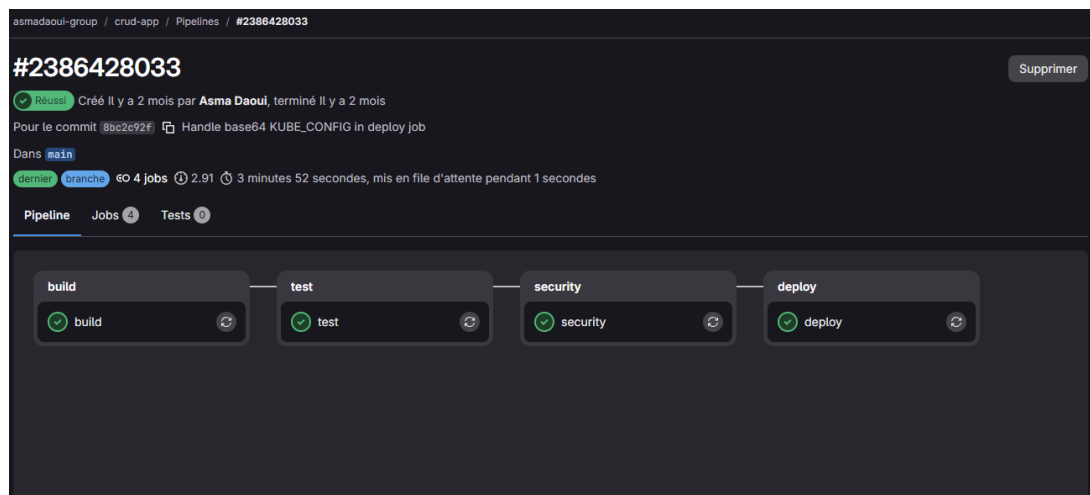


FIGURE 1 – Pipeline GitLab CI/CD — 4 stages : build, test, security, deploy

Pipeline en 4 stages : **build** (image Docker pushee dans GitLab Registry), **test** (Jest avec MongoDB service), **security** (scan Trivy), **deploy** (kubectl via runner Windows).

Listing 5 – .gitlab-ci.yml

```
stages:
  - build
  - test
  - security
  - deploy

variables:
  IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
  IMAGE_LATEST: $CI_REGISTRY_IMAGE:latest
  KUBE_NAMESPACE: crud-app

# BUILD
build:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
```

```

before_script:
  - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD
    $CI_REGISTRY
script:
  - docker build -t $IMAGE -t $IMAGE_LATEST .
  - docker push $IMAGE
  - docker push $IMAGE_LATEST
only:
  - main

# TEST
test:
  stage: test
  image: node:20-alpine
  services:
    - name: mongo:7.0
      alias: mongo
  variables:
    MONGO_URI: mongodb://mongo:27017/testdb
  script:
    - npm ci
    - npm test -- --passWithNoTests
  only:
    - main
    - merge_requests

# SECURITY - Trivy scan
security:
  stage: security
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
    TRIVY_AUTH_URL: $CI_REGISTRY
    TRIVY_USERNAME: $CI_REGISTRY_USER
    TRIVY_PASSWORD: $CI_REGISTRY_PASSWORD
  before_script:
    - apk add --no-cache curl
    - curl -sL https://raw.githubusercontent.com/aquasecurity/trivy/
      main/contrib/install.sh | sh -s -- -b /usr/local/bin
  script:
    - trivy image --exit-code 0 --severity HIGH,CRITICAL
      $CI_REGISTRY_IMAGE:latest
  only:
    - main

# DEPLOY - Windows runner -> Docker Desktop Kubernetes
deploy:
  stage: deploy
  tags:
    - windows
  before_script:
    - $decoded = [System.Text.Encoding]::UTF8.GetString(
      [System.Convert]::FromBase64String($env:KUBE_CONFIG))
    - $decoded | Out-File -FilePath "$env:TEMP\kubeconfig.yaml" -

```

```

    Encoding utf8
- $env:KUBECONFIG = "$env:TEMP\kubeconfig.yaml"
script:
- kubectl create namespace $env:KUBE_NAMESPACE
  --dry-run=client -o yaml | kubectl apply -f -
- kubectl create secret docker-registry gitlab-registry
  --docker-server=$env:CI_REGISTRY
  --docker-username=$env:CI_REGISTRY_USER
  --docker-password=$env:CI_REGISTRY_PASSWORD
  --namespace=$env:KUBE_NAMESPACE
  --dry-run=client -o yaml | kubectl apply -f -
- (Get-Content k8s/deployment.yaml) -replace
  'registry.gitlab.com/asmadaoui-group/crud-app:latest',
  $env:IMAGE | Set-Content k8s/deployment.yaml
- kubectl apply -f k8s/ --namespace=$env:KUBE_NAMESPACE
- kubectl rollout status deployment/crud-app
  --namespace=$env:KUBE_NAMESPACE --timeout=120s
only:
- main

```

5.1. Variable CI/CD requise

Variable	Type	Description
KUBE_CONFIG	Variable	kubeconfig encode en base64

Listing 6 – Generer KUBE_CONFIG (PowerShell)

```

[Convert]::ToBase64String(
[System.IO.File]::ReadAllBytes("$env:USERPROFILE\.kube\config")
)

```

5.2. GitLab Runner — Installation Windows

```

# Telecharger gitlab-runner.exe puis dans PowerShell admin :
.\gitlab-runner.exe install
.\gitlab-runner.exe start
.\gitlab-runner.exe register
# URL      : https://gitlab.com
# Executor: shell
# Tag      : windows

```

6. Terraform AWS — terraform/main.tf

Fichier separe pour creer une instance EC2 sur AWS avec Docker. L'instance pull automatiquement l'image depuis GitLab Registry et lance l'app + MongoDB au demarrage via user_data.

Listing 7 – terraform/main.tf

```

terraform {
  required_version = ">= 1.5"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Security Group
resource "aws_security_group" "crud_sg" {
  name          = "crud-app-sg"
  description   = "Allow HTTP and SSH"

  ingress {
    from_port    = 22
    to_port      = 22
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  ingress {
    from_port    = 3000
    to_port      = 3000
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }
}

# EC2 Instance
resource "aws_instance" "crud_app" {
  ami              = "ami-0c02fb55956c7d316"
  instance_type    = "t3.micro"
  vpc_security_group_ids = [aws_security_group.crud_sg.id]

  user_data = <<-EOF
  #!/bin/bash
  yum update -y
  amazon-linux-extras install docker -y
  service docker start
  usermod -a -G docker ec2-user

  # Login GitLab Registry
  docker login registry.gitlab.com \

```

```

    -u TON_GITLAB_USERNAME \
    -p TON_GITLAB_TOKEN

# MongoDB
docker network create crud-network
docker run -d --name mongo --network crud-network mongo:7.0

# App
docker run -d -p 3000:3000 --name crud-app \
  --network crud-network \
  -e MONGO_URI=mongodb://mongo:27017/cruddb \
  registry.gitlab.com/asmadaoui-group/crud-app:latest
EOF

tags = {
  Name = "crud-app-server"
}
}

output "instance_ip" {
  value      = aws_instance.crud_app.public_ip
  description = "IP publique de l'instance"
}

output "app_url" {
  value = "http://${aws_instance.crud_app.public_ip}:3000/health"
}

```

6.1. Commandes Terraform

```

# Configurer AWS CLI
aws configure
# Access Key ID      : <ta cle>
# Secret Access Key : <ta cle secrete>
# Default region    : us-east-1
# Output format     : json

# Verifier la connexion
aws sts get-caller-identity

# Dans le dossier terraform/
terraform init
terraform plan
terraform apply

# Verifier l'IP de l'instance
terraform output

# App disponible sur
# http://<instance_ip>:3000/health
# http://<instance_ip>:3000/items

# Detruire l'infrastructure
terraform destroy -auto-approve

```

7. Endpoints API

Method	Route	Action	Status
GET	/health	Health check	200
POST	/items	Creer un item	201
GET	/items	Lister tous les items	200
GET	/items/:id	Recuperer un item	200
PUT	/items/:id	Modifier un item	200
DELETE	/items/:id	Supprimer un item	200

Listing 8 – Exemple CRUD avec curl

```
# CREATE
curl -X POST http://localhost:3000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Item 1", "description": "Test"}'

# READ ALL
curl http://localhost:3000/items

# UPDATE (remplacer ID)
curl -X PUT http://localhost:3000/items/ID \
  -H "Content-Type: application/json" \
  -d '{"name": "Item modifie"}'

# DELETE
curl -X DELETE http://localhost:3000/items/ID
```

8. Architecture globale

Composant	Technologie	Port
API REST	Node.js + Express	3000
Base de donnees	MongoDB	27017
Container	Docker (multi-stage)	—
Cluster local	Docker Desktop K8s	30080
Registry	GitLab Container Registry	—
Pipeline CI/CD	GitLab CI (4 stages)	—
Cloud	AWS EC2 (Terraform)	3000